



INFORME PROYECTO GYM IRONSTRONG FITNESS

Gym IronStrong Fitness



En esta nueva etapa del proyecto, se dará continuidad a la implementación del sistema mediante una transición integral desde su base previa hacia una arquitectura de microservicios distribuida y por capas. A diferencia de una arquitectura tradicional de estilo monolítico, la cual centraliza y acopla todo el código fuente en una única unidad de despliegue, este enfoque se fundamenta en la división del ecosistema en componentes autónomos y modulares

El desarrollo actual contempla la integración de 6 nuevos microservicios, alcanzando un total de 11 microservicios operativos y completamente independientes. Las nuevas unidades funcionales integradas de forma autónoma en sus respectivos servidores son:

1. **Microservicio de Ubicación**
2. **Microservicio de Sucursal**
3. **Microservicio de Empleado**
4. **Microservicio de Pago**
5. **Microservicio de Ficha Médica**
6. **Microservicio Authenticator**

Justificación Arquitectónica y Beneficios Técnicos

El despliegue independiente de estos componentes en servidores pequeños y especializados otorga al sistema dos ventajas arquitectónicas fundamentales de nivel empresarial:

- **Escalabilidad Granular:** Permite dimensionar y escalar los recursos de hardware de un servicio específico que experimente alta demanda, sin necesidad de replicar o sobredimensionar el resto de la aplicación
- **Alta Tolerancia a Fallos (Resiliencia):** Al eliminar el punto único de fallo característico de los monolitos, el aislamiento de procesos garantiza que la eventual caída o degradación de un microservicio no provoque un efecto dominó, manteniendo la disponibilidad y operatividad del resto del ecosistema

Nuevas Implementaciones Específicas de la Experiencia

Más allá de la lógica de negocio de los nuevos servicios, el núcleo técnico de esta experiencia se centrará en la incorporación de tres pilares fundamentales para el ciclo de vida del software moderno:

- **Documentación Centralizada con Swagger/OpenAPI:** Implementación de herramientas para la autogeneración de especificaciones interactivas de las APIs, facilitando las pruebas de endpoints y el consumo de servicios a través del API Gateway
- **Testing Automatizado:** Diseño e integración de pruebas (unitarias y/o de integración) destinadas a validar el comportamiento correcto de cada componente bajo entornos aislados antes de su puesta en producción
- **Autenticación y Seguridad Distribuida:** Incorporación de mecanismos de seguridad (gestionados mediante el microservicio *Authenticator*) para restringir el acceso a los endpoints y asegurar que la comunicación inter- servicio se realice bajo estrictos estándares de autorización

Diagrama Modelo de Datos Entidad Relacional

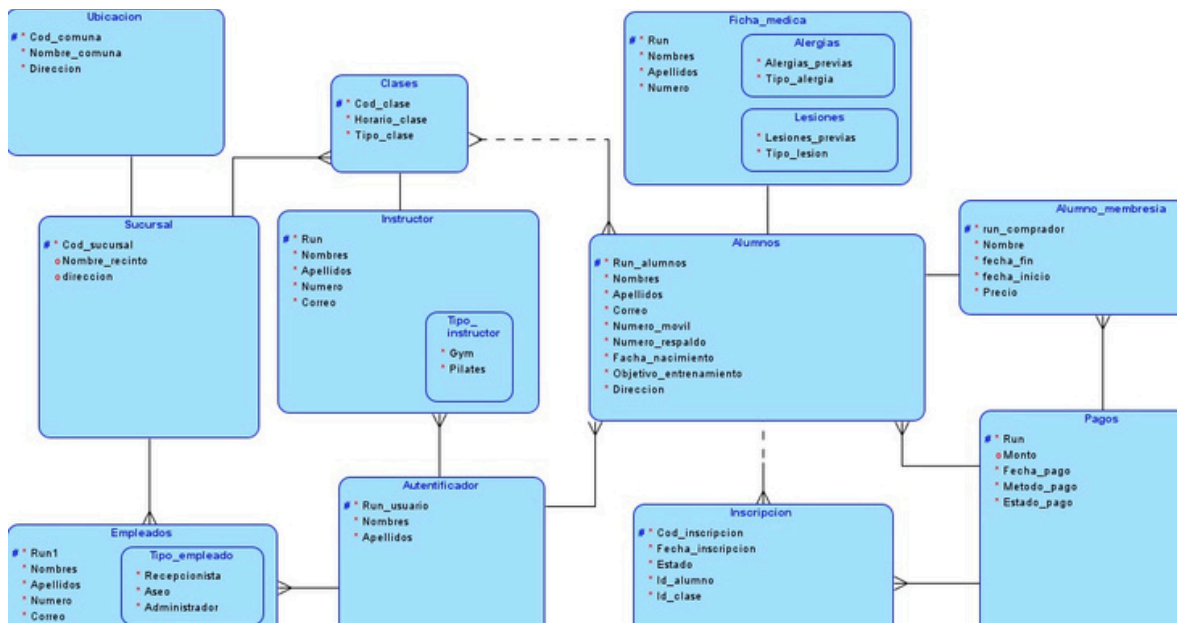
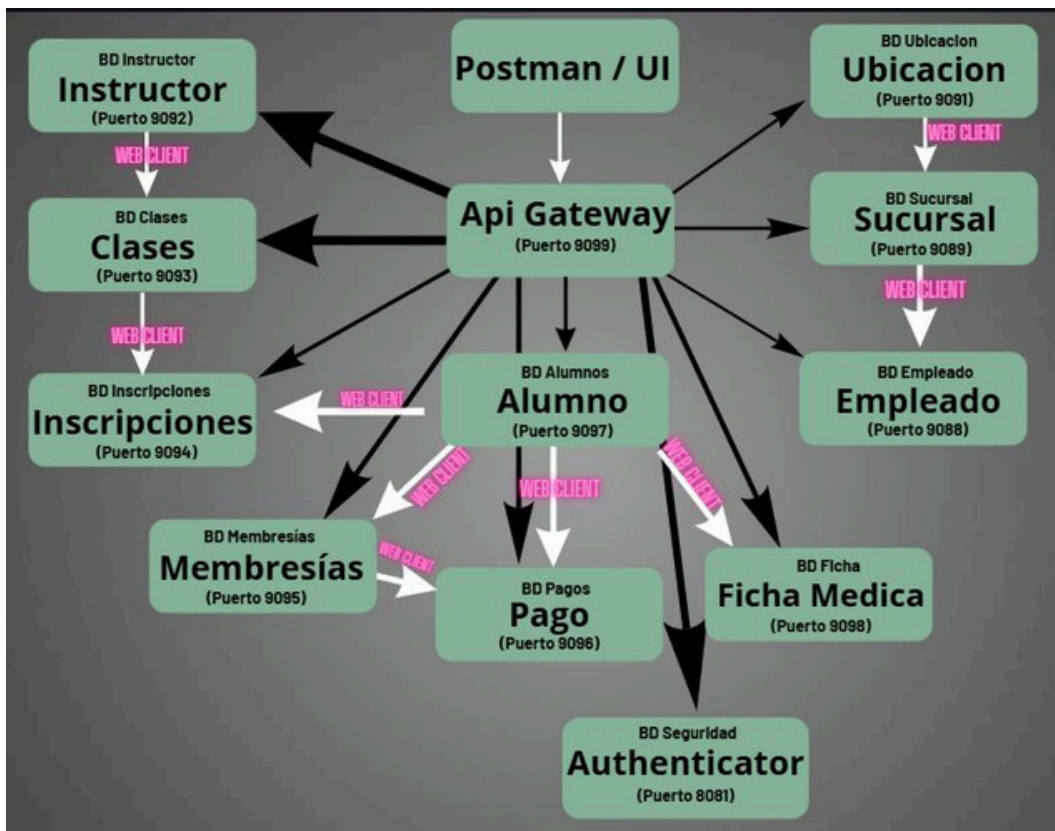


Diagrama Arquitectura de Microservicios



Servicio de autenticación

¿En qué consiste?

El servicio de autenticación es un mecanismo de defensa de ciberseguridad que verificara la identidad de los usuarios en los microservicios, antes de otorgarles acceso o permisos de entrada a un usuario que no tiene permitido realizar dicha acción

¿Cómo se implementa la autenticación por token?

Para proteger el ecosistema de 11 microservicios, se implementa una estrategia de seguridad centralizada y desacoplada que mitiga los riesgos de accesos no autorizados sin afectar el rendimiento del sistema. Esta se basa en tres componentes

1. Autenticación por Tokens (JWT)

- **Mecanismo:** Se descarta el uso de sesiones tradicionales en el servidor. El usuario inicia sesión enviando sus credenciales al microservicio *Authenticator*, el cual valida los datos y genera un token **JWT (JSON Web Token)** firmado criptográficamente que contiene su identidad y roles (ALUMNO, EMPLEADO, etc.)
- **Operación:** El cliente almacena este token y lo adjunta automáticamente en la cabecera de cada petición HTTP (Authorization: Bearer <token>) para identificarse de forma segura en las siguientes consultas

2. Protección de Rutas en el API Gateway

- **Control Centralizado:** El **API Gateway** actúa como un filtro perimetral único. Intercepta todas las peticiones entrantes dirigidas a las rutas de negocio (como pagos o fichas médicas)
- **Filtros de Seguridad:** El Gateway evalúa el token JWT: si es válido y está vigente, permite el reenvío de la petición al microservicio correspondiente; si es inexistente, inválido o está expirado, bloquea la comunicación inmediatamente devolviendo un estado **HTTP 401 Unauthorized**

3. Control de CORS (Cross-Origin Resource Sharing)

- **Definición:** Configuración alojada en el API Gateway para gestionar las restricciones del navegador web (*Same-Origin Policy*) cuando el frontend intenta consumir recursos del backend
- **Aplicación:** Permite declarar explícitamente qué dominios o URLs del cliente (ej. `http://localhost:9099`) son de confianza, autorizando el uso de métodos HTTP (GET, POST, etc.) y el intercambio seguro de cabeceras de autorización

La comunicación interservicio en este proyecto se resuelve de manera asíncrona y no bloqueante a través de WebClient (componente nativo de Spring WebFlux), optimizando el rendimiento frente a llamadas concurrentes

Su funcionamiento e implementación se sintetizan en:

- **Modelo Reactivo y Eficiente:** A diferencia de las herramientas tradicionales bloqueantes (como RestTemplate), WebClient no congela el hilo de ejecución mientras espera la respuesta del microservicio destino. La transferencia se gestiona mediante flujos de datos (Mono o Flux), permitiendo que el servidor de origen siga atendiendo otras solicitudes en paralelo
- **Intercambio Basado en Estándares:** Toda la mensajería interna viaja bajo el protocolo HTTP empleando estructuras JSON. Spring se encarga de serializar y deserializar automáticamente estos mensajes, traduciendo los objetos de dominio Java a texto y viceversa de forma transparente para la lógica de negocio

Swagger

¿Qué es?

Swagger (actualmente bajo la especificación OpenAPI) es un conjunto de herramientas de código abierto construido en torno a un lenguaje de descripción de interfaz (IDL) para describir, producir, consumir y visualizar servicios web RESTful. En términos sencillos, actúa como un contrato interactivo que traduce el código fuente de los endpoints de Java en una interfaz gráfica web dinámica, estandarizada y legible tanto por humanos como por computadoras.

¿En qué consiste su comportamiento en Microservicios?

En un entorno distribuido (como el sistema del gimnasio), cada microservicio independiente (Alumnos, Fichas Médicas, Ubicaciones, etc.) genera de forma interna su propia documentación técnica en formato JSON o YAML en la ruta estructurada /v3/api-docs.

Para evitar que el desarrollador o cliente tenga que abrir 11 pestañas distintas en el navegador (una para cada puerto), el API Gateway actúa como un Agregador de Swagger. El Gateway expone un único panel de Swagger UI unificado que intercepta y consume de forma asíncrona las definiciones OpenAPI de cada microservicio subyacente.

La problemática surge cuando la seguridad perimetral del Gateway malinterpreta estas consultas automáticas de metadatos como peticiones maliciosas anónimas, bloqueando la lectura del archivo de especificación (/v3/api-docs) con un error de acceso no autorizado y rompiendo el renderizado visual de la interfaz.

¿Cómo se implementa en un proyecto de microservicios con Spring Boot?

La implementación requiere enlazar la autogeneración local de cada microservicio con la exposición centralizada del API Gateway utilizando la dependencia Springdoc OpenAPI.

-Configuración de rutas en el API Gateway (application.yml)

Para que el agregador unificado de Swagger pueda recopilar los esquemas OpenAPI sin interferencias, se disocia el tráfico de negocio del tráfico de documentación dentro de las propiedades de enrutamiento del Gateway. Como ejemplo representativo del sistema, se observa cómo el microservicio de Alumnos posee dos mapeos distintos: el primero aplica el filtro de autenticación global (AuthenticationFilter) para resguardar los datos del negocio, mientras que el segundo (alumno-docs) expone el endpoint de metadatos de forma directa y libre de filtros interceptores

Asimismo, esta especificación descentralizada se replica de manera homóloga para el resto de los componentes del ecosistema (tales como instructor-docs, clases-docs, ficha-medica-docs, entre otros), los cuales alimentan la propiedad springdoc.swagger-ui.urls del Gateway para estructurar el menú desplegable unificado de la interfaz gráfica.

Testing

¿En qué consiste?

El Testing de Software es un proceso disciplinado y sistemático de evaluación que tiene como objetivo verificar y validar que una aplicación o sistema informático funciona exactamente según los requisitos técnicos y de negocio establecidos. Consiste en someter al código a diferentes condiciones y vectores de entrada para analizar su comportamiento, comparando los resultados obtenidos de forma real frente a los resultados esperados bajo un entorno controlado

¿Por qué es importante testear?

En el desarrollo de software moderno —y de manera crítica en sistemas basados en **microservicios**— el testing no es una etapa opcional, sino un pilar fundamental por las siguientes razones:

- **Detección Temprana de Errores (Reducción de Costos):** Encontrar un fallo en las fases iniciales de desarrollo cuesta una fracción de lo que costaría solucionarlo una vez que el sistema está desplegado en producción, donde un error puede corromper datos de base de datos o interrumpir el servicio
- **Facilidad de Refactorización y Mantenimiento:** Los sistemas evolucionan constantemente. Contar con una suite de pruebas robusta actúa como una red de seguridad. Si un desarrollador modifica un algoritmo para optimizarlo, las pruebas automáticas le indicarán de inmediato si rompió alguna funcionalidad existente en otra parte del código
- **Validación de Integraciones Complejas:** Al tener múltiples microservicios interactuando entre sí (por ejemplo, el servicio de Clases consultando asíncronamente al servicio de Instructores mediante WebClient), el testing garantiza que los contratos de datos y la comunicación reactiva respondan de forma resiliente ante fallas de red o respuestas inesperadas

¿En qué consisten las Pruebas Unitarias?

Las Pruebas Unitarias constituyen la base de la pirámide de pruebas en la ingeniería de software. Consisten en aislar y verificar la unidad lógica de código más pequeña e indivisible de una aplicación, la cual, en la programación orientada a objetos (como Java con Spring Boot), equivale a un método específico dentro de una clase de servicio

Las características esenciales de una prueba unitaria correcta son:

1. **Aislamiento Absoluto (Uso de Mocks):** Una prueba unitaria verdadera no debe conectarse a una base de datos real, ni levantar un servidor web, ni consumir un servicio externo real de internet. Para lograr esto, se utiliza el concepto de Mocking (a través de frameworks como Mockito). Las dependencias del servicio (como los repositorios JpaRepository o clientes de red) se reemplazan por objetos simulados cuyo comportamiento es estrictamente programado mediante instrucciones como `when(...).thenReturn(...)`
2. **Velocidad de Ejecución:** Debido a que operan completamente en memoria gracias al aislamiento de sus componentes, las pruebas unitarias se ejecutan en milisegundos. Esto permite correr cientos de pruebas en segundos dentro de tuberías de Integración Continua (CI/CD)
3. **Determinismo:** Una prueba unitaria debe ser completamente predecible. Independientemente de si se ejecuta de día, de noche, en una máquina local o en un servidor en la nube, si el código no ha cambiado, el resultado de la prueba debe ser exactamente el mismo
4. **Estructura Estandarizada (AAA):** Siguen rigurosamente el patrón arquitectónico de pruebas:
 - o **Arrange (Preparar):** Se instancian los objetos de prueba (como un InstructorDTO) y se configuran las respuestas de los componentes simulados
 - o **Act (Actuar):** Se ejecuta el método real bajo prueba (`clasesService.buscarPorId(...)`)
 - o **Assert (AfirmaryVerificar):** Se aplican criterios de validación (`assertNotNull`, `assertEquals`) para asegurar que el resultado retornado coincida de manera matemática y lógica con el comportamiento esperado de la regla de negocio

Alcances de Nuestra Implementacion en HTML

- Navegación fluida: Interfaz web completa con 6 secciones funcionales (Inicio, Presentación, Qué Ofrecemos, Productos, Membresías y Registro)
- Diseño adaptativo y visual: Estilo moderno en modo oscuro con colores neón/eléctricos optimizado mediante CSS
- Captura de datos de usuarios: Formulario funcional en la página de registro preparado para enviar la información de los nuevos alumnos
- Catálogo de productos: Exposición estructurada de productos de la tienda del gimnasio con imágenes, descripciones y precios

Lo que NO se puede hacer en el sistema

- Procesamiento de pagos en línea: La página muestra los planes y productos, pero no cuenta con una pasarela de pago real integrada
- Base de datos local en el navegador: El front-end no almacena datos por sí solo; si no hay conexión con el backend/microservicio, los datos del registro no se guardan permanentemente
- Carrito de compras dinámico: La sección de productos es puramente informativa/catálogo; no permite agregar elementos a un carrito ni calcular totales en tiempo real.
- Reserva de clases en vivo: Las clases se presentan como información estática, sin un sistema de agenda o cupos en tiempo real para las sucursales

Herramientas y Metodología de Desarrollo

Estructura Multi-Página (HTML):

Crea la estructura HTML semántica para 6 páginas independientes: index.html (Inicio), presentacion.html (Presentación), ofrecemos.html (Qué Ofrecemos), productos.html (Tienda/Productos), membresias.html (Planes) y registro.html (Formulario de Alumnos)

Implementa una barra de navegación (<nav>) en la cabecera que conecte todas las páginas mediante rutas relativas limpias para garantizar su correcto funcionamiento en servidores locales y remotos
En la sección de productos, incluye una cuadrícula con tarjetas de contenido que integren imágenes (), nombres, descripciones y precios

Diseño y Estilos Unificados (CSS):

Diseña una única hoja de estilos unificada (estilos.css) basada en una paleta de colores estilo Neón / Cyberpunk (Fondo negro #0b0b0b, tarjetas oscuras #141414, azul eléctrico #00f0ff y magenta #ff007f).

Aplica un diseño de botones para los enlaces del menú mediante display: inline-block con efectos de transición (transition) y resplandor al pasar el cursor (hover: box-shadow)

Remueve los contornos azules por defecto del navegador (outline: none) para mantener una experiencia limpia al interactuar con los elementos

Organiza los productos en una cuadrícula responsiva mediante CSS Grid (repeat(auto-fit, minmax(220px, 1fr))) y asegúrate de que el estado :hover de las tarjetas mantenga la legibilidad del texto utilizando un fondo gris oscuro sobrio (#222222)